

Sistemas Distribuidos
(Trimestre 26-P)

Lectura 2: Modelos de Comunicación

Dr. Francisco de Asís López Fuentes
Universidad Autonoma Metropolitana-Cuajimalpa
flopez@cua.uam.mx

Francisco A. López Fuentes

1

Introducción

La comunicación entre procesos es un factor clave para poder construir sistemas distribuidos, los paradigmas de comunicación más usados en sistemas distribuidos son:

- Cliente - Servidor
- Llamada a un procedimiento Remoto (RPC)
- Comunicación en grupo

Francisco A. López Fuentes

3

El Modelo Cliente - Servidor

- El término *Cliente - Servidor* (C-S) hace referencia a la comunicación en la que participan dos aplicaciones. Está basado en la comunicación de uno a uno.
- La aplicación que inicia la comunicación, enviando una petición y esperando una respuesta se llama cliente.
- Mientras que los servidores esperan pasivos, aceptan peticiones recibidas a través de la red, realizan el trabajo y regresan el resultado ó un código de error porque no fue hecho, al que genero la petición.

Francisco A. López Fuentes

5

Objetivo

Comprender los diferentes modelos de comunicación entre procesos más usados en el diseño de los sistemas distribuidos.

Agenda:

1. Introducción
2. Modelo cliente-servidor
3. Llamada a procedimiento remoto (RPC)
4. Comunicación en grupo
5. API de sockets

Francisco A. López Fuentes

2

Introducción (2)

Los conceptos fundamentales que deben ser considerados para la comunicación son:

- Los datos tienen que ser aplanados antes de ser enviados.
- Los datos tienen que ser representados de la misma manera en la fuente y destino.
- Los datos tienen que empaquetarse para ser enviados.
- Usar operaciones de *send* para enviar y *receive* para recibir.
- Especificar la comunicación, ya sea en modo bloqueante ó no bloqueante.
- Abstracción del mecanismo de paso de mensaje.
- La confiabilidad de la comunicación. Por ejemplo, preferir usar TCP en lugar de UDP.

Francisco A. López Fuentes

4

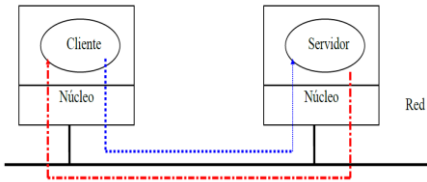
El Modelo Cliente - Servidor (2)

- Una máquina puede ejecutar un proceso ó varios procesos clientes.
- La transferencia de mensaje en el modelo C-S se ejecuta en el núcleo.
- Una operación general del funcionamiento del *Cliente - Servidor* se muestra en la figura, un servidor espera una petición sobre un puerto bien conocido, que ha sido reservado para un cierto servicio. Un cliente reserva un puerto arbitrario y no usado para poder comunicarse.

Francisco A. López Fuentes

6

El Modelo Cliente - Servidor (3)



Francisco A. López Fuentes

7

Características del software Cliente- Servidor

- El software cliente es un programa de aplicación que se vuelve cliente temporal cuando se necesita acceso remoto.
- El programa cliente llama directamente al usuario y se ejecuta sólo durante una sesión. Se ejecuta localmente en la computadora del usuario y se encarga de iniciar el contacto con el servidor.
- El programa cliente puede acceder a varios servicios y no necesita hardware especial ni un complejo sistema operativo.

Francisco A. López Fuentes

8

Características del software Cliente- Servidor

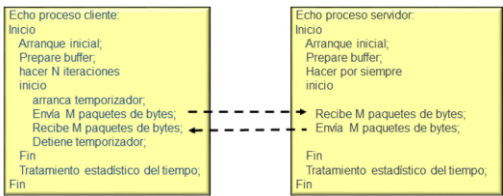
- El software servidor es un programa de propósitos especial dedicados a ofrecer un servicio, pero que puede manejar varios clientes remotos al mismo tiempo.
- El programa servidor se inicia automáticamente al arranque del sistema y continuará ejecutándose en varias sesiones. El servidor espera pasivamente el contacto de los clientes remotos.
- El software servidor por lo general necesita ser instalado en un sistema operativo más robusto y contar con bastante capacidad de recursos de hardware

Francisco A. López Fuentes

9

Características del software Cliente- Servidor

- Un pseudocódigo para un eco (echo) para un *Cliente – Servidor*



Francisco A. López Fuentes

10

- La comunicación *Cliente - Servidor* puede ser implementada a través de librerías PVM (Parallel Virtual Machine) o sockets

Deficiencias del Modelo Cliente- Servidor

- El paradigma de comunicación es la entrada/salida (E/S) ya que estos no son fundamentales en el cómputo centralizado.
- No permite la transparencia requerida para un ambiente distribuido.
- Un método en donde el programador no se preocupe de la transferencia de mensajes o de las E/S, es el RPC (llamada a un procedimiento remoto).

Francisco A. López Fuentes

11

Llamada de procedimiento remoto (RPC)

- La llamada de procedimiento remoto mejor conocido como RPC, Es una variante del paradigma *Cliente - Servidor* y es una vía para implementar en la realidad los protocolos del Cliente-Servidor.
- En el RPC un programa llama a un procedimiento localizado en otra máquina. El programador no se preocupa de las transferencias de mensajes o de las E/S.

Francisco A. López Fuentes

12

Llamada de procedimiento remoto (2)

- La idea de RPC es que una llamada a un procedimiento remoto se parezca lo más posible a una llamada local, lo que le permita una mayor transparencia.
- Para obtener esta transparencia el RPC usa un *resguardo de cliente*, que es el que se encarga de empaclar los parámetros en un mensaje y le solicita al núcleo que envíe el mensaje al servidor, posteriormente se bloquea hasta que regrese la respuesta.

Francisco A. López Fuentes

13

Modo de operación del RPC

1. El procedimiento cliente llama al resguardo del cliente de la manera usual.
2. El resguardo del cliente construye un mensaje y realiza un señalamiento al núcleo.
3. El núcleo cliente envía el mensaje al núcleo remoto
4. El núcleo remoto proporciona el mensaje al resguardo del servidor.
5. El resguardo del servidor desempaca los parámetros y llama al servidor.

Francisco A. López Fuentes

14

Modo de operación del RPC (2)

6. El servidor realiza el trabajo y regresa el resultado al resguardo
7. El resguardo del servidor empacla el resultado en un mensaje y hace un señalamiento mediante el núcleo.
8. El núcleo remoto envía el mensaje al núcleo del cliente.
9. El núcleo del cliente da el mensaje al resguardo del cliente.
10. El resguardo desempaca el resultado y lo entrega al cliente.

Francisco A. López Fuentes

15

Modo de operación del RPC (3)

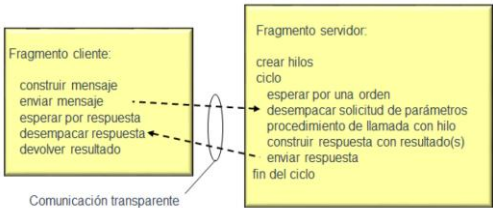


Francisco A. López Fuentes

16

Modo de operación del RPC (3)

Fragmentos de Cliente - Servidor en RPC



Francisco A. López Fuentes

17

Puntos problemáticos del RPC son

- Se deben de usar espacios de direcciones distintos, debido a que se ejecutan en computadoras diferentes.
- Como las computadoras pueden no ser idénticas la transferencia de parámetros y resultados puede complicarse.
- Ambas computadoras pueden descomponerse.

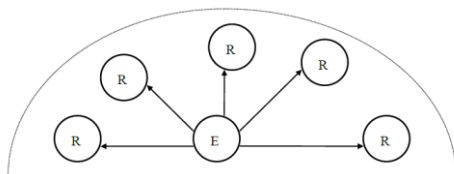
Francisco A. López Fuentes

18

Comunicación en Grupo

Generalidades:

- En un sistema distribuido puede haber comunicaciones entre procesos de uno a muchos como se indica en la Figura.



Francisco A. López Fuentes

19

Comunicación en Grupo (2)

- Los grupos son dinámicos, se pueden crear nuevos grupos y destruir grupos anteriores.
- Las técnicas para implantar la comunicación en grupos pueden ser:
 - Transmisión en multicast
 - Transmisión completa
 - Transmisión punto a punto.

Francisco A. López Fuentes

20

Comunicación en Grupo (3)

- Sobre los aspectos de diseño de los grupos, se tienen las siguientes opciones:
 - Grupos cerrados:* por ejemplo, procesamiento paralelo
 - Grupos abiertos:* como es el soporte de servidores redundantes
 - Grupo entre iguales (peers):* Permite que las decisiones se tomen en forma colectiva.
 - Ventajas: El grupo de compañeros es simétrico y no tiene punto de falla.
 - Desventaja: La toma de decisiones por votación es difícil, crea retraso y es costosa.

Francisco A. López Fuentes

21

Comunicación en Grupo (4)

- Grupo jerárquico:* Existe un coordinador y varios trabajadores.
- Membresía de grupo:*
 - Permite crear, eliminar grupos, agregar o eliminar procesos de grupos.
 - Utiliza técnicas como la del servidor del grupo o membresía distribuida.
 - Existen problemas de detección cuando un miembro ha fallado
- Direccionamiento al grupo:* los grupos deben poder direccionarse, estas pueden ser:
 - dirección única grupal
 - dirección de cada miembro del grupo
 - direccionamiento de predicados

Francisco A. López Fuentes

22

Comunicación en Grupo (5)

- Primitivas:* las primitivas de comunicación son:
 - send* y *receive*, de la misma forma que en una comunicación puntual
 - group_send* y *group_receive* para comunicación en grupo
- Atomicidad:* Se refiere a que cuando se envía un mensaje a un grupo, este debe llegarles a todos los miembros o a ninguno, así como garantizar la consistencia.

Francisco A. López Fuentes

23

Comunicación en Grupo (6)

- Ordenamiento de mensajes:* conjuntamente con la atomicidad permite que la comunicación en grupo sea fácil de comprender y utilizar, los criterios son:
 - ordenamiento con respecto al tiempo global
 - ordenamiento consistente
 - ordenamiento con respecto a grupos traslapados
- Escalabilidad:* permitir que el grupo continúe funcionando aun cuando se vayan agregando nuevos miembros a este.

Francisco A. López Fuentes

24

Interfaz de programación de aplicaciones (API) de sockets

- Las aplicaciones cliente servidor se valen de protocolos de transporte para comunicarse.
- Las aplicaciones deben de especificar los detalles sobre el tipo de servicio (cliente o servidor), los datos a transmitir y el receptor donde situar los datos.
- La interfaz entre un programa de aplicación y los protocolos de comunicación de un sistema operativo se llama interfaz de programación de aplicaciones (API).

Francisco A. López Fuentes

25

API de sockets (2)

- Un API define un conjunto de procedimientos que puede efectuar una aplicación al interactuar con el protocolo. Por lo general tiene procedimientos para cada operación básica.
- Los programadores pueden escoger una gran variedad de APIs para sistemas distribuidos, algunas de estas son: BSD socket, Sun's RPC/XDR, librería de paso de mensajes PVM y Windows Sockets.

Francisco A. López Fuentes

26

API de sockets (3)

- La API de *socket* que tuvo sus orígenes en el UNIX BSD, se volvió una norma de facto en la industria, y muchos sistemas operativos la han adoptado, tanto para computadoras personales (winsock) como para algunos sistemas UNIX.
- Un socket es un punto de referencia hacia donde los mensajes pueden ser enviados, o de donde pueden ser recibidos. Al llamar la aplicación a un procedimiento de socket, el control pasa a una rutina de la biblioteca de sockets que realiza las llamadas al sistema operativo para implementar la función de socket.

Francisco A. López Fuentes

27

API de sockets (4)

- UNIX BSD y los sistemas derivados de este contienen una biblioteca de sockets, la cual puede ofrecer a las aplicaciones una API de socket en un sistema de cómputo que no ofrece sockets originales.
- Los sockets emplean varios conceptos de otras partes del sistema, pero en particular están integrados con la E/S, por lo que una aplicación se comunica con un socket de la misma forma en que se transfiere un archivo.

Francisco A. López Fuentes

28

API de sockets (5)

- Cuando una aplicación crea un socket recibe un descriptor para hacer referencia a este.
- Si un sistema usa el mismo espacio de descriptor para los sockets y otras E/S es posible emplear la misma aplicación para transferir datos localmente como para su uso en red.
- La ventaja del método de socket es que la mayor parte de las funciones tienen tres o menos parámetros, sin embargo, se debe llamar a varias funciones al este método

Francisco A. López Fuentes

29

API de sockets (6)

Las principales funciones de la API sockets son:

- **socket()**
 - Esta rutina se usa para crear un socket y regresa un descriptor, correspondiente a este socket, este descriptor es usado en el lado del cliente y en el lado del servidor de su aplicación.
 - Desde el punto de vista de la aplicación, el descriptor de archivo es final de un canal de comunicación. La rutina retorna -1 si ocurre un error.

Francisco A. López Fuentes

30

API de sockets (7)

- **close()**
 - Indica al sistema que el uso de un socket debe ser finalizado. Si se usa un protocolo TCP (orientado a conexión), *close* termina la conexión antes de cerrarlo.
 - Cuando el socket se cierra, se libera al descriptor, por lo que la aplicación ya no transmite datos y el protocolo de transportación ya no acepta mensajes de entradas para el socket.
- **bind()**
 - Suministra un número una dirección local a asociar con el socket, ya que cuando un socket es creado, no cuenta con ninguna dirección.

Francisco A. López Fuentes

31

API de sockets (8)

- **listen()**
 - Esta rutina prepara un socket para aceptar conexiones. Sólo puede ser usado en sockets que usan un canal de comunicación virtual.
 - Esta rutina se deberá usar del lado del servidor de la aplicación antes de que se pueda aceptar alguna solicitud de conexión del lado del cliente.
 - Conforme las solicitudes de los clientes llegan toma un lugar en la cola. La cola de solicitudes permite que el sistema detenga las solicitudes nuevas mientras que el servidor se encarga de las actuales.

Francisco A. López Fuentes

32

API de sockets (9)

- **accept()**
 - Esta rutina es usada del lado servidor de la aplicación para permitir aceptar las conexiones de los programas clientes.
 - Después de configurar una cola de datos, el servidor llama *accept*, cesa su actividad y espera una solicitud de conexión de un programa cliente. Esta rutina sólo es válida en proveedores de transporte de circuito virtual.
 - Cuando llega una solicitud al socket especificado *accept* llena la estructura de la dirección, con la dirección del cliente que solicita la conexión y establece la longitud de la dirección.

Francisco A. López Fuentes

33

API de sockets (10)

- **accept()**
 - *accept* crea un socket nuevo para la conexión y regresa un descriptor al que lo invoca, este nuevo socket es usado por el servidor para comunicarse con el cliente, y al terminar es cerrado.
 - El socket original es usado por el servidor para aceptar la siguiente conexión del cliente.

Francisco A. López Fuentes

34

API de sockets (10)

- **connect()**
 - Esta rutina permite establecer una conexión a otro socket.
 - Se utiliza del lado del cliente de la aplicación, permitiendo que un protocolo TCP inicie una conexión en la capa de transporte para el servidor especificado.
 - Cuando se utiliza para protocolos sin conexión, esta rutina registra la dirección del servidor en el socket, permitiendo que el cliente transmita varios mensajes al mismo servidor.
 - Usualmente el lado cliente de la aplicación enlaza a una dirección antes de usar esta rutina, sin embargo, esto no es requerido.

Francisco A. López Fuentes

35

API de sockets (11)

- **send()**
 - Esta rutina es utilizada para enviar datos sobre un canal de comunicación tanto del lado del cliente como del lado servidor de la aplicación.
 - Se usa para socket orientados a conexión, sin embargo podría utilizarse para datagramas, pero haciendo uso de *connect()*, para establecer la dirección del socket.

Francisco A. López Fuentes

36

API de sockets (12)

- **sendto()**
 - Permite que el cliente ó servidor transmitan mensajes usando un socket sin conexión (usando datagramas).
 - Es exactamente similar a *send()*, sólo que se deberán especificar la dirección destino del socket al cuál se quiere enviar el dato.
 - Se puede usar en socket orientados a conexión, pero el sistema ignorar la dirección destino indicada en *sendto()*.

Francisco A. López Fuentes

37

API de sockets (13)

- **recv()**
 - Esta rutina lee datos desde un socket conectado y es usado tanto en el lado del cliente como del lado del servidor de la aplicación.
- **recvfrom()**
 - Esta rutina lee datos desde un socket sin conexión.
 - En este caso el sistema regresa la dirección del transmisor con los mensajes de entrada, permite registrar la dirección del socket transmisor, en la misma forma que espera *sendto()*, por lo que la aplicación usa la dirección registrada como destino de la respuesta.

Francisco A. López Fuentes

38

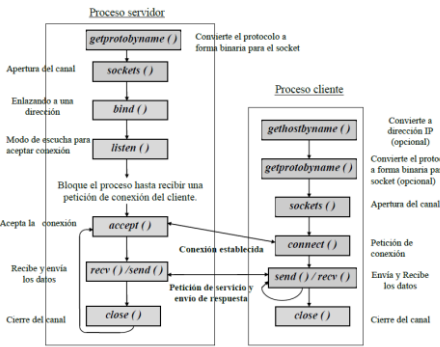
Servicio orientado a conexión

- Un servicio orientado a conexión requiere que dos aplicaciones establezcan una conexión de transportación antes de comenzar el envío de datos.
- Para establecer la comunicación ambas aplicaciones primero interactúan localmente con protocolo de transporte y después estos protocolos intercambian mensajes por la red.
- Una vez que ambos extremos estén de acuerdo y se haya establecido la conexión, las aplicaciones podrán enviar datos.

Francisco A. López Fuentes

39

Servicio orientado a conexión (1)



Francisco A. López Fuentes

40

Servicio orientado a conexión

- Después de llamar a *listen()* el servidor se pone en modo pasivo hasta que recibe una solicitud del cliente.
- Cuando la solicitud de servicio del cliente llega al socket monitoreado por la función *accept()*, se crea de modo automático un socket nuevo, que será conectado inmediatamente con el proceso cliente. Este socket se llama de servicio y se cierra al terminar la transferencia de datos.
- El socket original que recibió la solicitud de servicio permanece abierto en estado de escucha para seguir aceptando nuevas solicitudes.

Francisco A. López Fuentes

41

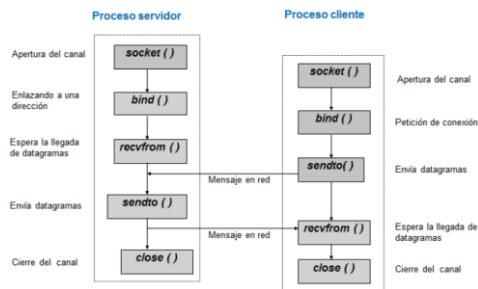
Comunicación sin conexión

- El servidor usa las llamadas a *socket()* y *bind()* para crear y unir un socket.
- Como el socket es sin conexión, se deberán usar las llamadas a *recvfrom()* y *sendto()*.
- Se llama a la función *bind()* pero no a *connect()*, dejando esta última como opcional.
- La dirección destino se especifica en la función *sendto()*.
- La función *recvfrom()* no espera conexión sino que responde a cualquier dato que llegue al puerto que tiene enlazado.

Francisco A. López Fuentes

42

Comunicación sin conexión



Francisco A. López Fuentes